

16.07 Dynamics

Problem Set 1 — Solutions

Due Tuesday, September 29

1 Spring-Loaded Inverted Pendulum Dynamics

Place the origin at the pivot and let $\hat{p}_r$ point from the pivot along the rod to the mass, and $\hat{p}_\theta$ point in the direction of increasing θ (i.e. $\hat{p}_\theta = d\hat{p}_r/d\theta$). Let $\hat{e}_x, \hat{e}_y, \hat{e}_z$ be the fixed Cartesian unit vectors, with $\hat{e}_x$ horizontal, $\hat{e}_y$ vertically up, and $\hat{e}_z = \hat{e}_x \times \hat{e}_y$ out of the page. With θ measured from the upward vertical as shown in the figure,

$$\hat{p}_r = \sin \theta \hat{e}_x + \cos \theta \hat{e}_y, \quad \hat{p}_\theta = \cos \theta \hat{e}_x - \sin \theta \hat{e}_y, \quad \dot{\hat{p}}_r = \dot{\theta} \hat{p}_\theta, \quad \dot{\hat{p}}_\theta = -\dot{\theta} \hat{p}_r.$$

The mass moves on a circle of fixed radius ℓ about the pivot, so its position, velocity, and acceleration are

$$\mathbf{r} = \ell \hat{p}_r, \quad \dot{\mathbf{r}} = \ell \dot{\theta} \hat{p}_\theta, \quad \mathbf{a} = \ddot{\mathbf{r}} = -\ell \dot{\theta}^2 \hat{p}_r + \ell \ddot{\theta} \hat{p}_\theta.$$

This is exactly the standard-pendulum kinematics from class; only the force balance below differs.

(a) $\vec{F} = m\vec{a}$ and the sign of gravity

Gravity is $\mathbf{F}_g = -mg \hat{e}_y$. Resolving into polar components using $\hat{p}_r, \hat{p}_\theta$ above,

$$\mathbf{F}_g \cdot \hat{p}_r = -mg \cos \theta, \quad \mathbf{F}_g \cdot \hat{p}_\theta = -mg(-\sin \theta) = mg \sin \theta.$$

Sign check against the figure: for $\theta > 0$ as drawn (mass displaced to the right of vertical), $\hat{p}_\theta$ points further to the right/down, in the sense of *increasing* θ . The tangential component $mg \sin \theta > 0$ for $0 < \theta < \pi$, meaning gravity pushes the mass in the direction of larger θ — i.e. gravity tends to topple the pendulum further away from vertical, which is exactly the expected (destabilizing) behavior of an inverted pendulum. This is the opposite sign convention from a standard hanging pendulum (where, measuring θ from the *downward* vertical, gravity is restoring, $-mg \sin \theta$); here θ is measured from the upward vertical, which flips the sign.

(b) Force equivalent to the spring torque

The torsion spring applies a restoring torque $\tau_{\text{spring}} = -k\theta$ about the pivot (negative of the angular displacement, zero at $\theta = 0$ as stated). Because this torque only has to balance the tangential dynamics of the mass about the pivot, its effect can be represented as an equivalent force $\mathbf{F}_s$ applied at the mass, radius ℓ from the pivot, purely in the $\hat{p}_\theta$ direction:

$$\mathbf{F}_s = F_s \hat{p}_\theta, \quad \tau_{\text{spring}} = \ell F_s \implies F_s = \frac{\tau_{\text{spring}}}{\ell} = -\frac{k\theta}{\ell}.$$

(The minus sign means $\mathbf{F}_s$ points in the $-\hat{p}_\theta$ direction whenever $\theta > 0$: it opposes the displacement, as a restoring spring should.)

(c) Why $\vec{\tau} = \vec{r} \times \vec{F}$ is (and isn't) invertible here

In general, $\vec{\tau} = \mathbf{r} \times \mathbf{F} = r\hat{p}_r \times (F_r\hat{p}_r + F_\theta\hat{p}_\theta) = rF_\theta \hat{e}_z$, since $\hat{p}_r \times \hat{p}_r = 0$. The torque depends *only* on the tangential component F_θ ; the radial component F_r drops out of the cross product entirely. So given only a torque τ , there are infinitely many forces $\mathbf{F} = (\tau/r)\hat{p}_\theta + F_r\hat{p}_r$ consistent with it, one for every choice of F_r .

It *is* invertible in this particular problem because the spring's effect can be represented as an equivalent force at the mass, and the natural (minimal) choice is a purely tangential force, $F_r = 0$: a radial component would do no work and would be absorbed into the rod's own axial constraint force. With F_r fixed at 0 by this choice, $F_\theta = \tau/\ell$ is the unique remaining unknown, and the inversion in part (b) is exact.

(d) Assembling the equation of motion

Newton's second law, $m\mathbf{a} = \mathbf{F}_g + \mathbf{F}_s + (\text{rod's axial constraint force})$, has no tangential contribution from the rod (axial forces lie along $\hat{p}_r$), so the $\hat{p}_\theta$ component reads

$$m(\ell\ddot{\theta}) = mg \sin \theta - \frac{k\theta}{\ell}.$$

Multiplying through by ℓ gives the equation of motion:

$$m\ell^2 \ddot{\theta} = mg\ell \sin \theta - k\theta \quad \Longleftrightarrow \quad \ddot{\theta} = \frac{g}{\ell} \sin \theta - \frac{k}{m\ell^2} \theta.$$

(The $\hat{p}_r$ component, $-m\ell\dot{\theta}^2 = -mg \cos \theta + F_r^{\text{rod}}$, just determines the axial rod force and is not needed for the dynamics.)

2 Linearization and Stability

For small θ , $\sin \theta \approx \theta$, so

$$m\ell^2 \ddot{\theta} = (mg\ell - k)\theta \quad \implies \quad \ddot{\theta} + a\theta = 0, \quad \boxed{a = \frac{k}{m\ell^2} - \frac{g}{\ell}}$$

(a) Natural frequency

For $a > 0$ the solutions are $\theta(t) = A \cos \omega_n t + B \sin \omega_n t$ with

$$\boxed{\omega_n = \sqrt{a} = \sqrt{\frac{k}{m\ell^2} - \frac{g}{\ell}}}$$

(b) Stability and critical stiffness

The sign of a determines the character of the solutions to $\ddot{\theta} + a\theta = 0$:

- $a > 0$ ($k > mg\ell$): as in part (a), the solutions are sinusoidal, $\theta(t) = A \cos \omega_n t + B \sin \omega_n t$. This is bounded oscillation about $\theta = 0$: **stable** (marginally, since there is no damping).
- $a < 0$ ($k < mg\ell$): the solutions are exponential, $\theta(t) = Ae^{\sqrt{|a|}t} + Be^{-\sqrt{|a|}t}$. Generic initial conditions excite the growing mode, giving exponential divergence from $\theta = 0$: **unstable**.

- $a = 0$ ($k = mg\ell$): the equation reduces to $\ddot{\theta} = 0$, with solutions $\theta(t) = \theta(0) + \dot{\theta}(0)t$ — linear growth (or a constant) rather than oscillation or exponential blowup. Linear analysis alone is inconclusive about the true (nonlinear) stability of this case (see Sec. 3(e)).

The transition occurs where the spring stiffness exactly balances the destabilizing gravitational “negative stiffness” $mg\ell$:

$$k_{cr} = mg\ell$$

3 Energy Analysis

(a) Kinetic and potential energy

The mass moves on a circle of radius ℓ with speed $\ell\dot{\theta}$. Measuring gravitational potential from the pivot height (height of the mass $= \ell \cos \theta$) and using the torsion spring energy $\frac{1}{2}k\theta^2$:

$$T(\theta, \dot{\theta}) = \frac{1}{2}m\ell^2\dot{\theta}^2, \quad U(\theta) = mg\ell \cos \theta + \frac{1}{2}k\theta^2.$$

(b) Plots of $U(\theta)$

With $m = \ell = 1$, $g = 9.8$: $k_{cr} = mg\ell = 9.8$, so the three cases are $k = 4.9, 9.8, 14.7$ and $U(\theta) = 9.8 \cos \theta + \frac{1}{2}k\theta^2$. The plots (with equilibria marked as described in (c) and (d)) are in Fig. 1; the code is in the Appendix.

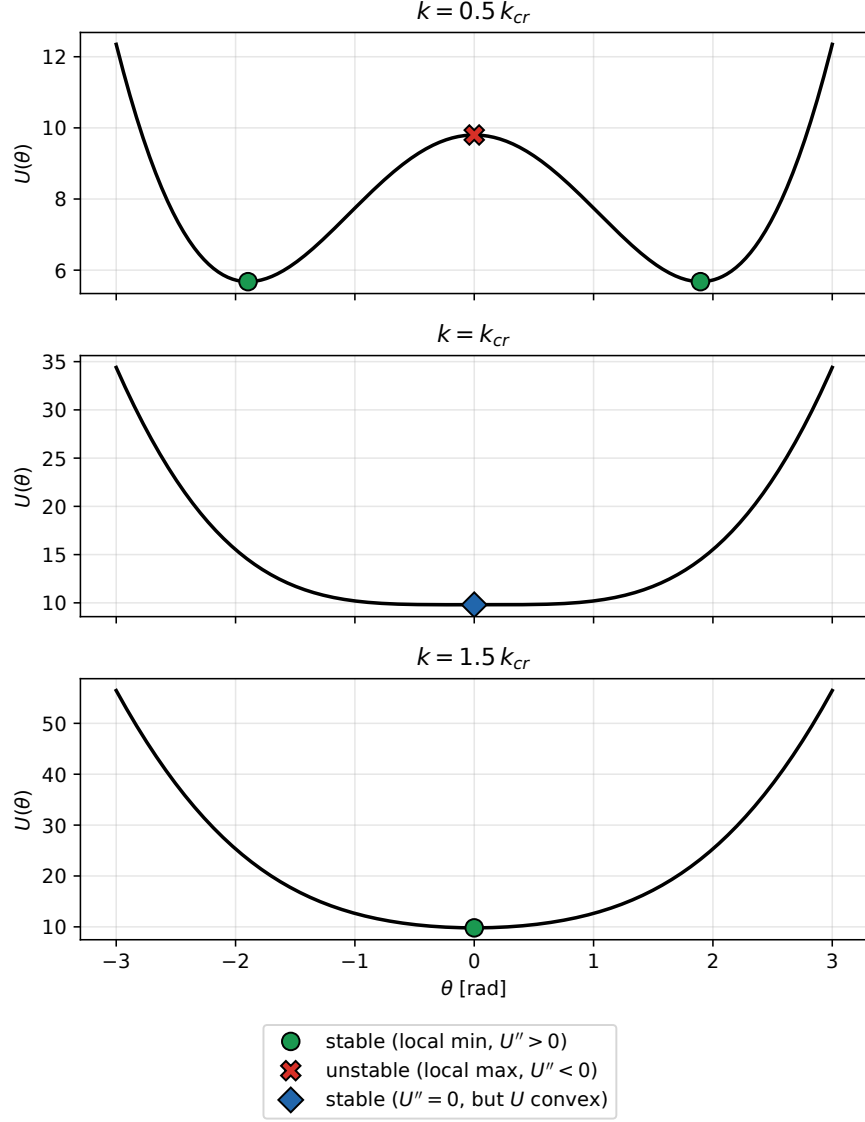


Figure 1: $U(\theta)$ for $k = 0.5 k_{cr}$, k_{cr} , and $1.5 k_{cr}$ ($m = \ell = 1$, $g = 9.8$). Green circles: stable equilibria (local minima). Red X: unstable equilibrium (local maximum). Blue diamond: equilibrium at $k = k_{cr}$ where $U''(0) = 0$; U is convex everywhere in this case, so $\theta = 0$ is still a (global) minimum and hence stable — see Sec. 3(e).

(c) Equilibria

An equilibrium is a point where the generalized force vanishes, $-dU/d\theta = 0$. On the plot this is a **point of zero slope** of $U(\theta)$. Here

$$\frac{dU}{d\theta} = -mg\ell \sin \theta + k\theta = 0 \quad \Longleftrightarrow \quad \theta = 0 \quad \text{or} \quad \frac{\sin \theta}{\theta} = \frac{k}{k_{cr}}.$$

- $\theta = 0$ (upright) is always an equilibrium.
- $\sin \theta/\theta$ decreases monotonically from 1 (at $\theta \rightarrow 0$) to 0 (at $\theta = \pi$), and $|\sin \theta/\theta| \leq 1/|\theta|$. So for $k \geq k_{cr}$ there is no other root, while for $k < k_{cr}$ there is exactly one pair $\pm\theta^*$ in $(0, \pi)$.

- For $k = 0.5 k_{cr}$: solving $\sin \theta^*/\theta^* = 0.5$ numerically gives $\theta^* \approx 1.8955$ rad. (Note $\theta^* > \pi/2$: the mass has swung below the horizontal.)

Note that $\theta = \pi$ (hanging straight down) is *not* an equilibrium here, because the torsion spring is wound by π there and applies torque $k\pi \neq 0$.

Case	Equilibria in $[-3, 3]$	$U''(\theta_e)$	Type
$k = 0.5 k_{cr}$	0	-4.90	unstable (max)
	± 1.8955	+8.03	stable (min)
$k = k_{cr}$	0	0	min, but $U'' = 0$ (see (e))
$k = 1.5 k_{cr}$	0	+4.90	stable (min)

These are marked in Fig. 1.

(d) Stability from $U(\theta)$

Linearizing about any equilibrium θ_e with $\theta = \theta_e + \delta$ gives

$$m\ell^2 \ddot{\delta} + U''(\theta_e) \delta = 0, \quad U''(\theta) = k - mg\ell \cos \theta,$$

the same form as the linearized equation of Sec. 2 with $a = U''(\theta_e)/(m\ell^2)$. So stability corresponds directly to the **local curvature of U** at the equilibrium:

- $U''(\theta_e) > 0$: U is **convex** (curves up) around θ_e , like the bottom of a valley: the equilibrium is **stable**.
- $U''(\theta_e) < 0$: U is **concave** (curves down) around θ_e , like the top of a hill: the equilibrium is **unstable**.
- $U''(\theta_e) = 0$: the curvature vanishes and this simple test is inconclusive; part (e) resolves this case.

The stable frequencies here are $\omega_n = \sqrt{4.9} = 2.21$ rad/s at $\theta = 0$ for $k = 1.5 k_{cr}$, and $\sqrt{8.03} = 2.83$ rad/s at $\pm\theta^*$ for $k = 0.5 k_{cr}$. The markers in Fig. 1 are color-coded accordingly (green = stable, red = unstable, blue = the borderline $k = k_{cr}$ case, resolved below).

(e) The critical case $k = k_{cr}$

Setting $k = mg\ell$, the second derivative of the potential is

$$U''(\theta) = k - mg\ell \cos \theta = mg\ell(1 - \cos \theta) \geq 0 \quad \text{for every } \theta,$$

with equality only at $\theta = 0 \pmod{2\pi}$. So U is convex everywhere, not just near $\theta = 0$: the potential curves upward around every point. By the same curvature criterion as part (d), a potential that curves upward around $\theta = 0$ means the upright equilibrium is **stable** — this holds even though the curvature happens to vanish exactly at $\theta = 0$ itself, since convexity of the whole well around that point is enough to confirm it sits at the bottom. Note that we have proved this analytically above, but making this argument in some form with reference to the plot is enough for full credit.

Why the linearization fails. At $k = k_{cr}$, $a = 0$, so the quadratic term in the Taylor expansion of U about $\theta = 0$ vanishes:

$$U(\theta) = mg\ell \left(\cos \theta + \frac{1}{2}\theta^2 \right) = mg\ell \left(1 + \frac{\theta^4}{24} - \dots \right).$$

With the θ^2 term gone, the next term in the expansion — the quartic term $mg\ell \theta^4/24$ — dominates the local shape of U near $\theta = 0$. Its coefficient is positive, so U still curves upward near the origin and the equilibrium is stable, but this is information the quadratic (linearized dynamics) approximation simply cannot see: whenever the linear term vanishes, the next-order term in the Taylor expansion takes over, and reading off stability requires looking at that nonlinear term instead. Equivalently, in the equation of motion, expanding $\sin \theta$ gives $\ddot{\theta} \approx -\frac{g}{6\ell}\theta^3$ near the origin: a nonlinear, but still restoring, term. (Consequence: oscillation period depends on amplitude, $T \propto 1/A$, unlike a linear oscillator.)

4 Numerical Simulation

Setup. With state $\mathbf{x} = (\theta, \dot{\theta})^T$, the dynamics take the form $\dot{\mathbf{x}} = f(\mathbf{x})$ with

$$f(\mathbf{x}) = \begin{bmatrix} \dot{\theta} \\ \frac{g}{\ell} \sin \theta - \frac{k}{m\ell^2} \theta \end{bmatrix}.$$

Here are solutions integrated with `scipy.integrate.solve_ivp` (RK45, the same Dormand–Prince pair as `ode45`), using `rtol`= 10^{-10} , `atol`= 10^{-12} , on $t \in [0, 60]$ s. For each $k \in \{0.5, 1, 1.5\}k_{cr}$, the same batch of $N = 20$ runs was integrated with $\dot{\theta}(0) = 0$ and $\theta(0) \sim U(-3, 3)$ rad. (Random seed 2, chosen as the first seed that gave a batch with at least one above-barrier run for the $k = 0.5k_{cr}$ separatrix discussion; only about 7% of the sampling range produces that.) As a sanity check, energy is conserved to a relative drift below 4×10^{-9} for all runs, so the integration is accurate.

(a)–(b) Phase-plane plots

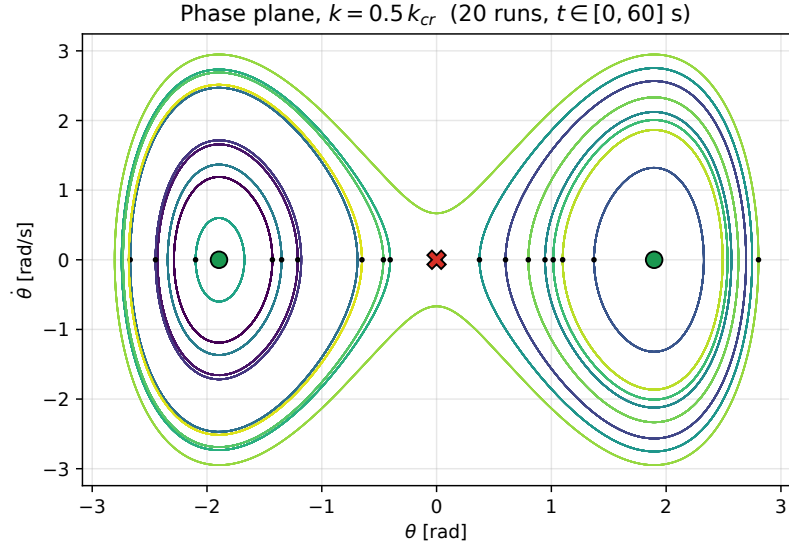


Figure 2: Phase plane, $k = 0.5 k_{cr}$. Black dots: initial conditions. Green circles: stable equilibria $\pm\theta^*$; red X: unstable equilibrium at the origin.

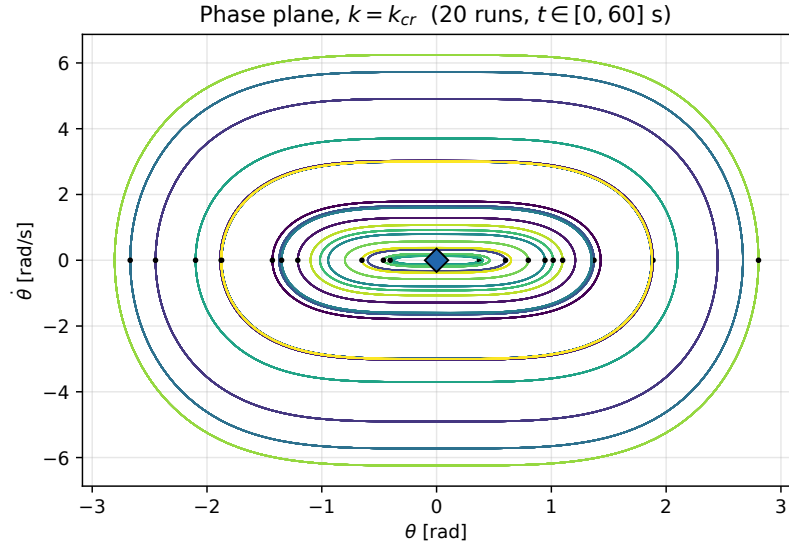


Figure 3: Phase plane, $k = k_{cr}$ (blue diamond: the $k = k_{cr}$ equilibrium, $U''(0) = 0$ but stable by convexity — Sec. 3(e)).

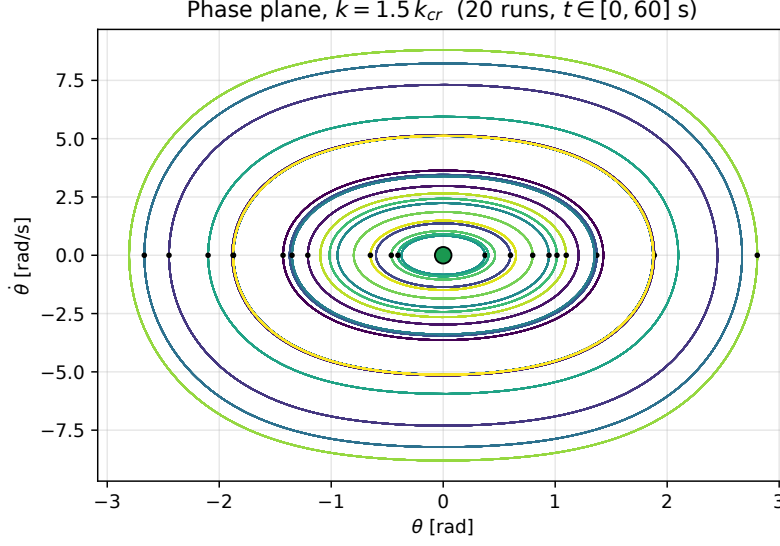


Figure 4: Phase plane, $k = 1.5 k_{cr}$.

Observations. Every trajectory is a closed curve, since each is a level set of the conserved energy E . Nothing spirals into or out of an equilibrium because there is no damping.

- $k = 1.5 k_{cr}$ (Fig. 4): closed loops around the origin, elliptical near the origin with $\dot{\theta}_{\max}/\theta_{\max} \approx \omega_n = 2.21 \text{ s}^{-1}$, becoming more rounded at large amplitude. The origin is a center: stable, as predicted ($a = 4.9 > 0$).
- $k = k_{cr}$ (Fig. 3): this is the case the linearization could not settle. The orbits are again closed loops around the origin. This confirms the nonlinear stability conclusion of Sec. 3(e), and the equilibrium is *stable but not asymptotically stable*.
- $k = 0.5 k_{cr}$ (Fig. 2): the origin is a saddle (unstable); runs starting on either side of it are pushed away from it and circulate around one of the two stable equilibria $\pm\theta^*$. One run with $\theta(0) = 2.80$ has enough energy to pass over the hump and encloses both wells.

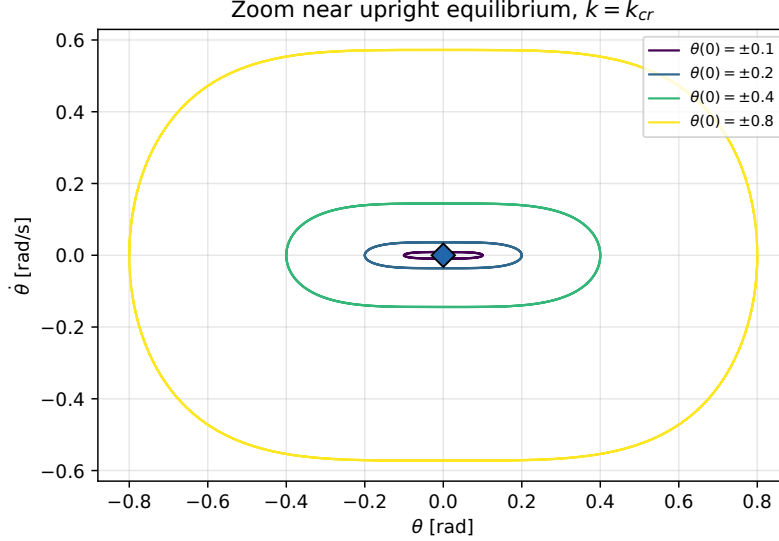


Figure 5: $k = k_{cr}$: small-amplitude runs ($\theta(0) = \pm 0.1, \pm 0.2, \pm 0.4, \pm 0.8$), each integrated for 1.5 estimated periods. The orbits stay arbitrarily close to the origin if started close, confirming the stability predicted in Sec. 3(e) from the convexity of U .

(c) Separatrix for $k = 0.5 k_{cr}$

As the problem statement notes, there is no damping here, so trajectories don't actually converge to $\pm\theta^*$ – they orbit forever – and strictly speaking there are no basins of attraction. There is still a well-defined *separatrix*: the curve that divides initial conditions that remain confined to oscillate about $-\theta^*$, about $+\theta^*$, or (if they carry enough energy) swing past both.

Finding it as a trajectory (per the hint). Since $E = \frac{1}{2}m\ell^2\dot{\theta}^2 + U(\theta)$ is conserved, every trajectory is a level curve of E , and the separatrix is *itself a trajectory*: the one passing through the saddle (unstable equilibrium) at the origin, with energy

$$E_{sep} = U(0) = mg\ell = 9.8.$$

Generating it numerically requires an initial condition that lies exactly on this level curve. The saddle itself, $(\theta, \dot{\theta}) = (0, 0)$, is a fixed point, so the curve's other zero-velocity point is used instead: since U is even and $U(0) = mg\ell$, the equation $U(\theta_{\max}) = U(0)$ has a second root $\theta_{\max} > 0$,

$$mg\ell \cos \theta_{\max} + \frac{1}{2}k\theta_{\max}^2 = mg\ell \quad \xrightarrow{m=\ell=1, k=4.9} \quad \theta_{\max} \approx 2.783 \text{ rad}.$$

Releasing the pendulum from rest at $\theta(0) = \theta_{\max}$, $\dot{\theta}(0) = 0$ puts it exactly on the separatrix (zero velocity $\Rightarrow E_0 = U(\theta_{\max}) = E_{sep}$). Integrating forward in time traces one half of the right-hand loop; integrating backward in time traces the other half. The trajectory only reaches the saddle asymptotically as $t \rightarrow \pm\infty$, so a long but finite integration ($t = \pm 400$ s) traces the loop essentially all the way around: the simulated curve passes within 1.2×10^{-6} rad of the saddle, with energy conserved to 2×10^{-10} . The left-hand loop can be found by starting at $-\theta_{\max}$ instead of $\theta_{\max}$.

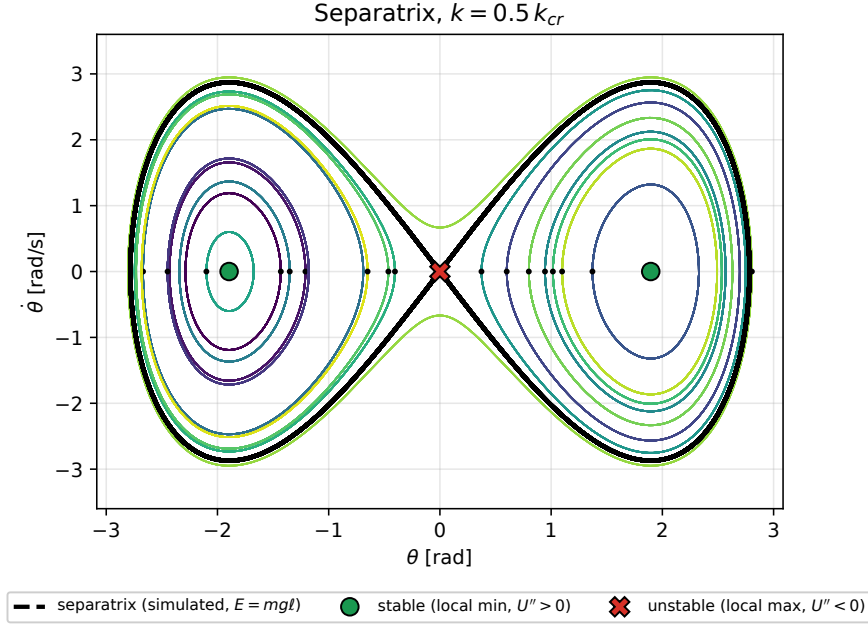


Figure 6: Phase plane for $k = 0.5 k_{cr}$ with the separatrix (dashed black), obtained by simulating a trajectory released from rest at $\theta = \theta_{\max} = 2.783$. Runs inside the left loop circulate around $-\theta^*$, runs inside the right loop around $+\theta^*$; the one run outside (started at $\theta(0) = 2.80$) encloses both loops.

Because the batch's initial conditions have $\dot{\theta}(0) = 0$, $E_0 = U(\theta_0)$, and:

- $|\theta_0| < \theta_{\max}$ (with $\theta_0 \neq 0$): $E_0 < E_{sep}$, so the run is confined to oscillate in one well. The sign of θ_0 selects the well. Sample: 11 runs in the left well, 8 in the right well.
- $|\theta_0| > \theta_{\max}$: $E_0 > E_{sep}$, so the run swings past the hump and orbits both wells. Sample: 1 run (with $\theta_0 = 2.80$).

No trajectories cross the separatrix.

A Code

The script below generates all figures and numbers quoted above.

```

1  """16.07 Problem Set 1 -- spring-loaded inverted pendulum.
2  Generates all figures and numbers used in the solutions document.
3  """
4  import json
5  import numpy as np
6  import matplotlib
7  matplotlib.use("Agg")
8  import matplotlib.pyplot as plt
9  from scipy.integrate import solve_ivp
10 from scipy.optimize import brentq
11
12 plt.rcParams.update({"font.size": 10, "axes.grid": True, "grid.alpha": 0.3,
13                     "pdf.fonttype": 42})
14
15 # ----- parameters

```

```

16 m, l, g = 1.0, 1.0, 9.8
17 k_cr = m * g * l
18 ratios = [0.5, 1.0, 1.5]
19 ks = [r * k_cr for r in ratios]
20 labels = [r"$k=0.5\,k_{cr}$", r"$k=k_{cr}$", r"$k=1.5\,k_{cr}$"]
21
22 GREEN, RED, BLUE = "#1a9850", "#d73027", "#2166ac"
23
24
25 def U(th, k):
26     return m * g * l * np.cos(th) + 0.5 * k * th**2
27
28
29 def Upp(th, k):
30     return k - m * g * l * np.cos(th)
31
32
33 def energy(th, thd, k):
34     return 0.5 * m * l**2 * thd**2 + U(th, k)
35
36
37 def rhs(t, y, k):
38     th, thd = y
39     return [thd, (g / l) * np.sin(th) - k / (m * l**2) * th]
40
41
42 def equilibria(k):
43     """Return list of (theta_eq, kind) in [-3,3]."""
44     eq = [(0.0, None)]
45     r = k / k_cr
46     if r < 1:
47         ts = brentq(lambda x: np.sin(x) / x - r, 1e-3, np.pi)
48         eq += [(ts, None), (-ts, None)]
49     out = []
50     for th, _ in eq:
51         u2 = Upp(th, k)
52         if abs(u2) < 1e-9:
53             kind = "degenerate"
54         elif u2 > 0:
55             kind = "stable"
56         else:
57             kind = "unstable"
58         out.append((float(th), kind, float(u2)))
59     return out
60
61
62 results = {"k_cr": k_cr}
63
64 # ----- Fig 1: U(theta), 3(b-d)
65 th = np.linspace(-3, 3, 1201)
66 fig, axs = plt.subplots(3, 1, figsize=(6.4, 8.2), sharex=True)
67 for ax, k, lab, r in zip(axs, ks, labels, ratios):
68     ax.plot(th, U(th, k), "k", lw=1.8)
69     for te, kind, u2 in equilibria(k):
70         if kind == "stable":
71             ax.plot(te, U(te, k), "o", ms=9, color=GREEN, mec="k", zorder=5)
72         elif kind == "unstable":
73             ax.plot(te, U(te, k), "X", ms=10, color=RED, mec="k", zorder=5)
74         else:
75             ax.plot(te, U(te, k), "D", ms=9, color=BLUE, mec="k", zorder=5)
76     ax.set_ylabel(r"$U(\theta)$")
77     ax.set_title(lab)
78 axs[-1].set_xlabel(r"$\theta$ [rad]")
79 from matplotlib.lines import Line2D
80 handles = [
81     Line2D([], [], marker="o", ls="", ms=9, color=GREEN, mec="k", label="stable (local min, $U'>0$)",
82     Line2D([], [], marker="X", ls="", ms=10, color=RED, mec="k", label="unstable (local max,

```

```

    '$U''<0$)'),
83     Line2D([], [], marker="D", ls="", ms=9, color=BLUE, mec="k", label="stable ($U''=0$, but
    '$U$ convex)'),
84 ]
85 fig.legend(handles=handles, loc="lower center", ncol=1, frameon=True, bbox_to_anchor=(0.5,
    -0.0))
86 fig.tight_layout(rect=(0, 0.09, 1, 1))
87 fig.savefig("fig_potential.pdf")
88 plt.close(fig)
89
90 results["equilibria"] = {f"{r}": equilibria(k) for r, k in zip(ratios, ks)}
91
92 # separatrix quantities for k = 0.5 k_cr
93 k05 = 0.5 * k_cr
94 th_star = [e for e in equilibria(k05) if e[0] > 0][0][0]
95 E_sep = U(0.0, k05) # = m g l
96 th_max = brentq(lambda x: 2 * m * g * l * (1 - np.cos(x)) - k05 * x**2, 2.0, 3.0)
97 results.update({"theta_star": th_star, "E_sep": E_sep, "theta_max": th_max,
98               "Upp_star": float(Upp(th_star, k05))})
99
100 # ----- simulations
101 N = 20
102 # Draw theta0 ~ U(-3,3); use the first seed whose batch contains at least one
103 # above-barrier run (|theta0| > theta_max ~ 2.78, only ~7% of the range) so that
104 # all three qualitative orbit types appear in the k = 0.5 k_cr plot.
105 for SEED in range(200):
106     th0s = np.random.default_rng(SEED).uniform(-3, 3, N)
107     if (np.sum(np.abs(th0s) > th_max) >= 1 and np.sum(th0s > 0) >= 6 and np.sum(th0s < 0) >=
        6
108         and np.sum(np.abs(th0s) < 0.3) == 0):
109         break
110 results["seed"] = SEED
111 t_end = 60.0
112 t_eval = np.linspace(0, t_end, 12001)
113 sols = {}
114 stats = {}
115 for r, k in zip(ratios, ks):
116     trajs = []
117     for th0 in th0s:
118         s = solve_ivp(rhs, (0, t_end), [th0, 0.0], args=(k,), method="RK45",
119                       t_eval=t_eval, rtol=1e-10, atol=1e-12)
120         trajs.append(s.y)
121     sols[r] = np.array(trajs) # (N, 2, T)
122     Y = sols[r]
123     E = energy(Y[:, 0], Y[:, 1], k)
124     drift = np.max(np.abs(E - E[:, :1]) / np.maximum(np.abs(E[:, :1]), 1.0))
125     stats[r] = {"max_rel_energy_drift": float(drift),
126               "max_abs_theta": float(np.max(np.abs(Y[:, 0])))}
127 results["sim"] = stats
128 results["theta0"] = [float(x) for x in th0s]
129
130 # ----- Fig 2: phase planes, 4(b)
131 cmap = plt.get_cmap("viridis")
132 def eq_markers(ax, k):
133     for te, kind, _ in equilibria(k):
134         if kind == "stable":
135             ax.plot(te, 0, "o", ms=9, color=GREEN, mec="k", zorder=6)
136         elif kind == "unstable":
137             ax.plot(te, 0, "X", ms=10, color=RED, mec="k", zorder=6)
138         else:
139             ax.plot(te, 0, "D", ms=9, color=BLUE, mec="k", zorder=6)
140
141 for r, k, lab, name in zip(ratios, ks, labels, ["05", "10", "15"]):
142     fig, ax = plt.subplots(figsize=(6.2, 4.4))
143     Y = sols[r]
144     for i in range(N):
145         ax.plot(Y[i, 0], Y[i, 1], lw=0.9, color=cmap(i / (N - 1)))
146         ax.plot(Y[i, 0, 0], Y[i, 1, 0], ".", color="k", ms=4)

```

```

147     eq_markers(ax, k)
148     ax.set_xlabel(r"$\theta$ [rad]")
149     ax.set_ylabel(r"$\dot{\theta}$ [rad/s]")
150     ax.set_title(f"Phase plane, {lab} ({N} runs, $t$ in [0, {t_end:.0f}] s)")
151     fig.tight_layout()
152     fig.savefig(f"fig_phase_{name}.pdf")
153     plt.close(fig)
154
155 # ----- Fig 3: k = k_cr zoom near origin with small ICs
156 fig, ax = plt.subplots(figsize=(6.2, 4.4))
157 small = [0.1, 0.2, 0.4, 0.8]
158 for j, a in enumerate(small):
159     for sgn in (+1, -1):
160         # integrate for ~1.5 quartic-well periods
161         c = g / (6 * l)
162         T_est = 7.416 / (a * np.sqrt(c))
163         s = solve_ivp(rhs, (0, 1.5 * T_est), [sgn * a, 0.0], args=(k_cr,),
164                       method="RK45", rtol=1e-11, atol=1e-13,
165                       t_eval=np.linspace(0, 1.5 * T_est, 4000))
166         ax.plot(s.y[0], s.y[1], lw=1.2, color=cmap(j / (len(small) - 1)),
167               label=rf"$\theta(0)=\pm{a}$" if sgn == 1 else None)
168 ax.plot(0, 0, "D", ms=9, color=BLUE, mec="k", zorder=6)
169 ax.set_xlabel(r"$\theta$ [rad]"); ax.set_ylabel(r"$\dot{\theta}$ [rad/s]")
170 ax.set_title(r"Zoom near upright equilibrium, $k=k_{cr}$")
171 ax.legend(loc="upper right", fontsize=8)
172 fig.tight_layout(); fig.savefig("fig_phase_cr_zoom.pdf"); plt.close(fig)
173
174 # ----- separatrix as a simulated trajectory
175 # (Sec. 4(c) hint): the separatrix passes through the saddle at theta=0 and has
176 # a turning point (theta_dot=0) at theta = +/-theta_max, since U(theta_max)=U(0).
177 # Starting at rest exactly at theta_max puts the IC exactly on the separatrix;
178 # integrating forward/backward traces the loop (it only reaches the saddle
179 # asymptotically as t -> +/-infinity, so a finite-time run traces the loop
180 # almost all the way around, slowing to a crawl near the saddle).
181 t_sep = 400.0
182 s_f = solve_ivp(rhs, (0, t_sep), [th_max, 0.0], args=(k05,), method="RK45",
183               rtol=1e-12, atol=1e-14, t_eval=np.linspace(0, t_sep, 20000))
184 s_b = solve_ivp(rhs, (0, -t_sep), [th_max, 0.0], args=(k05,), method="RK45",
185               rtol=1e-12, atol=1e-14, t_eval=np.linspace(0, -t_sep, 20000))
186 sep_right = np.concatenate([s_b.y[:, ::-1], s_f.y], axis=1)
187 sep_left = np.array([-sep_right[0], sep_right[1]]) # mirror symmetry theta -> -theta
188 results["separatrix_sim_check"] = {
189     "closest_rad_to_saddle_fwd": float(np.min(np.hypot(s_f.y[0], s_f.y[1]))),
190     "max_abs_E_minus_Esep_along_sim": float(np.max(np.abs(
191         energy(s_f.y[0], s_f.y[1], k05) - E_sep)))
192 }
193
194 # ----- Fig 4: separatrix, k = 0.5 k_cr, 4(c)
195 def sep_speed(x):
196     val = 2 * (g / l) * (1 - np.cos(x)) - k05 / (m * l**2) * x**2
197     return np.sqrt(np.clip(val, 0, None))
198
199 xs = np.linspace(-th_max, th_max, 2001)
200 Y = sols[0.5]
201 E = energy(Y[:, 0], Y[:, 1], k05)
202 sign_E = np.sign(E[:, 0] - E_sep)
203 # does sign(E - E_sep) ever change along a trajectory? (should not)
204 sign_changes = int(np.sum([np.any(np.sign(E[i] - E_sep) != sign_E[i]) for i in range(N)]))
205 # geometric check: inside the loop <=> |thd| < sep_speed(theta) for |theta| < th_max
206 inside_left = inside_right = outer = 0
207 geom_violations = 0
208 margin_min = np.inf
209 for i in range(N):
210     th_i, thd_i = Y[i, 0], Y[i, 1]
211     m_in = np.abs(th_i) < th_max
212     if E[i, 0] < E_sep:
213         if th_i[0] < 0: inside_left += 1
214         else: inside_right += 1
215     # must stay strictly inside loop and on the same side of theta=0

```

```

215         geom_violations += int(np.any(np.abs(thd_i) >= sep_speed(th_i) + 1e-9) or np.any(np.
            sign(th_i) != np.sign(th_i[0])))
216     else:
217         outer += 1
218         # must stay strictly outside: |thd| > sep_speed wherever the separatrix exists
219         geom_violations += int(np.any(np.abs(thd_i[m_in]) <= sep_speed(th_i[m_in]) - 1e-9))
220         margin_min = min(margin_min, float(np.min(np.abs(E[i] - E_sep))))
221     results["separatrix_check"] = {"left_well": inside_left, "right_well": inside_right,
222                                   "outer": outer, "sign_changes": sign_changes,
223                                   "geom_violations": int(geom_violations),
224                                   "min_abs_E_minus_Esep": margin_min}
225
226 fig, ax = plt.subplots(figsize=(6.4, 4.8))
227 for i in range(N):
228     ax.plot(Y[i, 0], Y[i, 1], lw=0.9, color=cmap(i / (N - 1)), alpha=0.9)
229     ax.plot(Y[i, 0, 0], Y[i, 1, 0], ".", color="k", ms=4)
230 # separatrix obtained by directly simulating a trajectory released from rest
231 # at the turning point theta = +- theta_max (Sec. 4(c) hint)
232 ax.plot(sep_right[0], sep_right[1], "k--", lw=2.2, label="separatrix (simulated)")
233 ax.plot(sep_left[0], sep_left[1], "k--", lw=2.2)
234 eq_markers(ax, k05)
235 ax.set_xlabel(r"$\theta$ [rad]"); ax.set_ylabel(r"$\dot{\theta}$ [rad/s]")
236 ax.set_title(r"Separatrix, $k=0.5$, $k_{cr}$")
237 ax.legend(handles=[Line2D([], [], color="k", ls="--", lw=2.2, label=r"separatrix (simulated,
    $E=mg\ell$)"] + handles[:2],
238             loc="upper center", bbox_to_anchor=(0.5, -0.16), ncol=3, fontsize=8, framealpha
                =0.95)
239 ax.set_ylim(-3.6, 3.6)
240 fig.tight_layout(); fig.savefig("fig_separatrix.pdf"); plt.close(fig)
241
242 with open("results.json", "w") as f:
243     json.dump(results, f, indent=2)
244     print(json.dumps(results, indent=2))

```